

1 Introduction to Operating Systems

An Operating System (OS) is system software that manages computer hardware, software resources, and provides common services for computer programs. It acts as an intermediary between users and hardware, ensuring efficient resource allocation, security, and usability. Without an OS, you'd have to write machine code for every task; with one, you can run applications seamlessly.

1.1 Key Roles of an OS

- **Resource Management:** Allocates CPU, memory, storage, and I/O devices.
- **User Interface:** Provides CLI (Command-Line Interface) or GUI (Graphical User Interface).
- **Security and Protection:** Prevents unauthorized access and ensures process isolation.
- **Error Handling:** Detects and recovers from faults.
- **Abstraction:** Hides hardware complexities (e.g., treating a hard drive as a file system).

1.2 Evolution and Importance

OSes evolved from batch processing in the 1950s to modern multitasking systems. They're essential in desktops (Windows, macOS), servers (Linux), mobiles (Android, iOS), embedded devices (RTOS like FreeRTOS), and cloud (container OSes like Container-Optimized OS). In 2025, OSes emphasize AI integration, security against quantum threats, and energy efficiency for edge computing.

2 History of Operating Systems

- **1950s–1960s:** Early mainframes used simple monitors (e.g., GM-NAA I/O). Multiprogramming emerged with IBM OS/360 (1964).
- **1970s:** Unix (Bell Labs, 1969) introduced portability and modularity. MS-DOS (1981) for PCs.
- **1980s–1990s:** GUI rise with Macintosh System (1984), Windows 3.0 (1990). Linux kernel by Linus Torvalds (1991).
- **2000s:** Mobile OSes like Symbian, then iOS (2007) and Android (2008). Virtualization with VMware.
- **2010s–2020s:** Cloud focus (e.g., AWS Linux). Windows 10/11, macOS shifts to ARM (Apple Silicon). Linux dominates servers (e.g., Ubuntu, Red Hat). Recent: AI-enhanced OSes (e.g., Windows Copilot in 2023+), post-quantum cryptography in kernels.
- **Current Trends (2025):** Hybrid OSes for quantum-classical computing, sustainable designs (e.g., low-power kernels), and privacy-focused like GrapheneOS.

3 Types of Operating Systems

OSes vary by design, purpose, and architecture.

Type	Description	Examples
Batch OS	Processes jobs in batches without user interaction.	Early IBM systems
Time-Sharing/Multitasking	Allows multiple users/programs to run concurrently via time slices.	Unix, Windows
Real-Time OS (RTOS)	Guarantees response within strict time limits (hard/soft real-time).	VxWorks, FreeRTOS
Distributed OS	Manages resources across networked computers as a single system.	Amoeba, Google Spanner OS
Network OS	Facilitates network resource sharing (not fully distributed).	Novell NetWare
Mobile OS	Optimized for touch, battery, sensors.	Android, iOS
Embedded OS	For specific devices, minimal footprint.	Embedded Linux, Windows IoT
Virtual OS/Hypervisors	Runs multiple OSes on one hardware (Type-1 bare-metal, Type-2 hosted).	VMware ESXi, Hyper-V

Table 1: Types of Operating Systems

- **Monolithic vs. Microkernel:** Monolithic (e.g., Linux) has all services in kernel space (fast but risky). Microkernel (e.g., MINIX) moves services to user space (modular, secure but slower).
- **32-bit vs. 64-bit:** 64-bit handles more memory (> 4GB) and is standard now.

4 OS Architecture and Kernel

The **kernel** is the core of the OS, handling low-level tasks.

4.1 Kernel Types

- **Monolithic Kernel:** All OS services in one address space.
- **Microkernel:** Minimal kernel; drivers/file systems as user processes.
- **Hybrid Kernel:** Combines both (e.g., Windows NT kernel).
- **Exokernel:** Gives apps direct hardware access for performance.

4.2 System Calls

- Interface between user programs and kernel (e.g., `open()`, `read()` for files).
- Invoked via interrupts or traps.

4.3 Boot Process

1. **BIOS/UEFI:** Firmware initializes hardware.
2. **Bootloader** (e.g., GRUB): Loads kernel.
3. **Kernel Initialization:** Sets up memory, devices.
4. **Init Process:** Starts user-space services (e.g., `systemd` in Linux).

5 Process Management

A **process** is a program in execution, with code, data, stack, and heap.

5.1 Process States

- New → Ready → Running → Waiting → Terminated.
- **PCB (Process Control Block)**: Stores PID, state, registers, etc.

5.2 Scheduling

- Allocates CPU time.
- **Algorithms**:
 - FCFS (First-Come, First-Served): Simple, but convoy effect.
 - SJF (Shortest Job First): Optimal for average wait, but starvation.
 - Priority: Higher priority first (can cause starvation; aging fixes).
 - Round-Robin: Time quanta (fair, for time-sharing).
 - Multilevel Queue/Feedback: Multiple queues for different priorities.
- **Preemptive vs. Non-Preemptive**: Can interrupt or not.
- **Multithreading**: Threads share process resources (lighter than processes). Models: Many-to-One, One-to-One, Many-to-Many.

5.3 Inter-Process Communication (IPC)

- **Shared Memory**: Fast, but needs synchronization.
- **Message Passing**: Pipes, queues (e.g., FIFO in Unix).
- **Synchronization**: Prevents race conditions.
 - **Mutex**: Binary semaphore for mutual exclusion.
 - **Semaphores**: Counting for resource control.
 - **Monitors**: High-level with condition variables.
- **Deadlocks**: Cycles in resource allocation. Prevention: Banker's Algorithm.

6 Memory Management

Manages RAM for processes.

6.1 Basics

- **Logical vs. Physical Address**: MMU (Memory Management Unit) translates.
- **Contiguous Allocation**: Fixed (partitions) or dynamic (fragmentation issues).
- **Paging**: Divides memory into pages/frames. Page table maps virtual to physical. Solves external fragmentation.
 - **Demand Paging**: Loads pages on demand (page faults).

- **Page Replacement:** FIFO, LRU (Least Recently Used), Optimal.
- **Segmentation:** Variable-sized segments (code, data). Can cause external fragmentation.
- **Virtual Memory:** Uses disk as extension (swap space). Thrashing: Excessive swapping.

6.2 Advanced

- **TLB (Translation Lookaside Buffer):** Cache for page table entries.
- **Memory Protection:** Access rights per page.

7 File Systems

Manages storage and data organization.

7.1 File Concepts

- **Attributes:** Name, type, size, permissions.
- **Operations:** Create, read, write, delete.
- **Directory Structure:** Hierarchical (tree), acyclic graph.

7.2 File System Types

- **FAT32:** Simple, cross-platform, limited.
- **NTFS:** Windows, supports journaling, encryption.
- **ext4:** Linux, efficient for large files.
- **APFS:** Apple, optimized for SSDs.
- **Distributed:** HDFS (Hadoop), for big data.

7.3 Implementation

- **Allocation Methods:** Contiguous, linked, indexed (inodes in Unix).
- **Free Space Management:** Bitmaps, linked lists.
- **Journaling:** Logs changes for crash recovery.

8 Input/Output (I/O) Management

Handles devices like keyboards, disks, networks.

8.1 I/O Hardware

- **Ports, Buses, Controllers:** Interface standards (USB, PCIe).
- **Polling vs. Interrupts:** Polling wastes CPU; interrupts efficient.
- **DMA (Direct Memory Access):** Bypasses CPU for large transfers.

8.2 I/O Software

- **Device Drivers:** Kernel modules for specific hardware.
- **Buffering/Spooling:** Caches data, manages print queues.
- **Disk Scheduling:** FCFS, SSTF (Shortest Seek Time First), SCAN (elevator).

9 Security and Protection

- **Authentication:** Passwords, biometrics, MFA.
- **Access Control:** ACLs (Access Control Lists), RBAC (Role-Based).
- **Encryption:** File/disk (e.g., BitLocker).
- **Firewalls/Antivirus:** In-kernel or user-space.
- **Vulnerabilities:** Buffer overflows, privilege escalation.
- **Modern:** SELinux (mandatory access), AppArmor. Post-2020: Zero-trust models, AI threat detection.

10 Virtualization and Containers

- **Virtualization:** Hypervisors create VMs (e.g., KVM in Linux).
- **Containers:** Lightweight (share kernel), e.g., Docker using namespaces/cgroups.
- **Benefits:** Isolation, scalability, portability.
- **Orchestration:** Kubernetes for managing containers.

11 Modern Operating Systems Examples

- **Windows 11 (2021+):** Hybrid work focus, Android app support via WSA, AI features like Copilot.
- **macOS Sonoma/Ventura+:** ARM-based, Continuity with iOS, enhanced privacy.
- **Linux Distributions:** Ubuntu (user-friendly), Fedora (cutting-edge), Arch (customizable). Kernel 6.x (2025) with better Rust support for drivers.
- **Android 15 (2024+):** Material You, privacy sandbox.
- **iOS 18+:** AI integrations (Apple Intelligence), RCS support.
- **Specialized:** Chrome OS (web-centric), QNX (automotive real-time).

12 Advanced Topics

- **Distributed Systems:** Consistency models (e.g., CAP theorem).
- **Cloud OS:** Serverless (e.g., AWS Lambda OS abstractions).
- **AI in OS:** Predictive resource allocation.
- **Quantum-Resistant OS:** Updated crypto libraries.
- **Performance Tuning:** Profiling tools (e.g., perf in Linux).
- **OS Development:** Tools like QEMU for emulation, GCC for compilation.

13 Best Practices and Learning Tips

- **Security First:** Regular updates, least privilege.
- **Efficiency:** Monitor with tools like Task Manager (Windows), top/htop (Linux).
- **Customization:** Shell scripting (Bash/PowerShell).
- **Practice:** Install Linux in VM, write simple kernel modules.